

Python for Web Development

Lesson 5: Database Integration with SQLAlchemy

Database Integration with SQLAlchemy - Study Notes

Key Concepts

- **Engine & Session** – Core objects that manage database connections and transactional state.
- **Declarative Base** – The pattern used to define ORM models as Python classes that map to tables.
- **CRUD Operations** – Create, Read, Update, Delete actions performed via the `Session` API.
- **Dialect Specific URLs** – Connection strings that tell SQLAlchemy which database driver (SQLite, PostgreSQL, etc.) to use.
- **Migrations (Alembic)** – Optional but common tool for evolving schema definitions over time.

Summary

SQLAlchemy provides a powerful, Pythonic interface for working with relational databases. At its heart is the **Engine**, which encapsulates a pool of DBAPI connections and is created from a database URL (e.g., `sqlite:///app.db` or `postgresql+psycopg2://user:pw@host/dbname`). From the Engine we obtain a **Session**, a short lived workspace that tracks object state and flushes changes to the database upon commit.

Models are defined by subclassing `declarative_base()` and declaring columns with `Column` types. Relationships are expressed via `relationship()` and foreign key constraints. Once models are in place, typical CRUD workflows involve:

1. **Create** – instantiate model objects, add them to the session, and commit.
2. **Read** – query the session using `session.query(Model).filter(...)` or the 2.0 style `select()` construct.
3. **Update** – modify attributes of persisted objects; the session tracks the changes and issues an `UPDATE` on commit.
4. **Delete** – call `session.delete(obj)` or issue a `delete()` statement, then commit.

SQLAlchemy abstracts away dialect differences, so the same model code works for SQLite (ideal for development/testing) and PostgreSQL (suited for production) with only the connection URL changing.

Important Points

- **Engine creation**: `create_engine(database_url, echo=True)` – `echo` logs SQL statements (helpful for debugging).
- **Session management**: Use `sessionmaker(bind=engine)` to get a Session class; prefer context managers (`with Session()` as `session:`) to ensure proper commit/rollback.
- **Declarative base**: `Base = declarative_base()`; all model classes inherit from `Base`.
- **Column types**: Import from `sqlalchemy` (e.g., `Integer`, `String`, `DateTime`, `ForeignKey`).
- **Relationships**: `relationship("RelatedModel", back_populates="owner")` enables easy navigation.
- **Query API**: 2.0 style (`select(Model).where(...)`) is preferred; legacy `query()` still works.

- **Transactions**: Changes are not persisted until `session.commit()`; `session.rollback()` aborts the transaction.
- **Engine disposal**: Call `engine.dispose()` when the application shuts down to close pooled connections.
- **Alembic**: Use `alembic revision --autogenerate` to create migration scripts from model changes.

Examples

Example 1 – SQLite Setup & Simple Model

```
python
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker
```

1p ã Engine (SQLite file)

```
engine = create_engine("sqlite:///example.db", echo=False)
```

2p ã Declarative base

```
Base = declarative_base()
```

3p ã Model definition

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String(50), nullable=False)
    email = Column(String(120), unique=True, nullable=False)
```

4p ã Create tables

```
Base.metadata.create_all(engine)
```

5p ã Session factory

```
Session = sessionmaker(bind=engine)
```

CRUD demo

```
with Session() as session:
```

```
    # Create
```

```
    new_user = User(name="Ada", email="ada@example.com")
```

```
    session.add(new_user)
```

```
    session.commit()
```

```
    # Read
```

```
    user = session.query(User).filter_by(email="ada@example.com").one()
```

```

print(user.name) # Ada
# Update
user.name = "Ada Lovelace"
session.commit()
# Delete
session.delete(user)
session.commit()
...

```

****Explanation:****

- The engine points to a local SQLite file (`example.db`).
- `Base.metadata.create\_all(engine)` issues the `CREATE TABLE` statements if they don't exist.
- A `Session` context manager guarantees that `commit()` is called on success or `rollback()` on exception.
- Standard ORM operations demonstrate the full CRUD cycle.

Example 2 – PostgreSQL Connection with Relationship

```

```python
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey
from sqlalchemy.orm import declarative_base, relationship, sessionmaker

```

## PostgreSQL URL (adjust credentials as needed)

```

engine = create_engine(
 "postgresql+psycopg2://myuser:mypassword@localhost:5432/mydb",
 echo=True, # show SQL for learning
 future=True # enable 2.0 style execution
)
Base = declarative_base()
class Department(Base):
 __tablename__ = "departments"
 id = Column(Integer, primary_key=True)
 name = Column(String(100), nullable=False, unique=True)
 employees = relationship("Employee", back_populates="department")
class Employee(Base):
 __tablename__ = "employees"
 id = Column(Integer, primary_key=True)
 name = Column(String(100), nullable=False)
 dept_id = Column(Integer, ForeignKey("departments.id"))
 department = relationship("Department", back_populates="employees")
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
with Session() as session:
 # Create a department and two employees

```

```

eng = Department(name="Engineering")
alice = Employee(name="Alice Smith", department=eng)
bob = Employee(name="Bob Jones", department=eng)
session.add_all([eng, alice, bob])
session.commit()
Read: eager load employees with joinedload to avoid N+1
from sqlalchemy.orm import joinedload
dept = session.query(Department).options(joinedload(Department.employees))\
 .filter_by(name="Engineering").one()
for emp in dept.employees:
 print(emp.name) # Alice Smith, Bob Jones
...

```

**\*\*Explanation:\*\***

- Uses PostgreSQL via the `psycopg2` driver; only the URL changes from the SQLite example.
- Demonstrates a one to many relationship (`Department.employees`).
- `joinedload` prevents the classic N+1 query problem when accessing related collections.
- The same model classes work unchanged; SQLAlchemy translates the appropriate SQL for PostgreSQL.

## Quick Review

1. What object is responsible for managing the connection pool and creating sessions?
2. How do you ensure that a session's changes are either committed or rolled back automatically?
3. Name two advantages of using the declarative base approach over manual `Table` definitions.
4. In the 2.0 style, which construct replaces the legacy `session.query()` for selecting rows?
5. Why is it recommended to use `engine.dispose()` at application shutdown?

## Further Reading

- **\*\*SQLAlchemy 2.0 Tutorial\*\*** – Official guide covering the new Core/ORM unification.
- **\*\*Alembic Documentation\*\*** – Best practices for schema migrations with SQLAlchemy.
- **\*\*PostgreSQL Specific Features\*\*** – Using `ARRAY`, `JSONB`, and range types via SQLAlchemy dialects.
- **\*\*Performance Tuning\*\*** – Connection pooling strategies, `lazyload` vs `joinedload`, and batch inserts.
- **\*\*Testing with SQLite in memory\*\*** – Techniques for fast unit tests using `sqlite:///memory:`.

---

\*These notes capture the essentials of configuring SQLite/PostgreSQL, defining models, and performing CRUD operations with SQLAlchemy. Use them as a quick reference while building data driven applications.\*

